

Assignment 8

Polymorphism in Java: Concepts, Types, and Implementation

Name: Akarsh Anand

Section: 2CSE14

Roll No.: 25SCS1003000760

Objective

The objective of this assignment is to understand the concept of Polymorphism, its different types, and the implementation of Method Overloading through practical Java programs.

Part A: Understanding of Theory Concept

1. Explain Polymorphism in Java

Definition: Polymorphism is one of the four fundamental principles of Object-Oriented Programming (OOP), derived from the Greek words “poly” (many) and “morphs” (forms).

Why Polymorphism is important in OOP

- Code Reusability
- Flexibility and Extensibility
- Simplifies Code Maintenance
- Supports Dynamic Behavior
- Enables Loose Coupling

2. Explain the Types of Polymorphism

Polymorphism in Java is broadly divided into two types:

(a) Compile-Time Polymorphism (Static Polymorphism)

Compile-time polymorphism is also known as static polymorphism because the method that needs to be executed is decided at compile time itself, based on the method signature (name, number, and type of parameters).

(b) Run-Time Polymorphism (Dynamic Polymorphism)

Run-time polymorphism is also known as dynamic polymorphism because the decision about which method to execute is made at run time, not compile time.

```
class Animal {
    void sound() { System.out.println("Animal makes a sound"); }
}
class Dog extends Animal {
    @Override
    void sound() { System.out.println("Dog barks"); }
}

// At run time:
Animal a = new Dog();
a.sound();    // Output: Dog barks
```

3. Explain Method Overloading

Definition

Method Overloading is a feature in Java that allows a class to have more than one method with the same name, provided that their parameter lists (number of parameters, type of parameters, or order of parameters) are different.

Rules of Method Overloading

- The method name must be the same.
- The parameter list must differ in at least one of the following ways: number of parameters, data type of parameters, or order/sequence of parameters.
- Overloaded methods can have different access modifiers.
- Overloaded methods can have different throws clauses.

Different Ways to Overload a Method

- By changing the number of parameters: `add(int a, int b)` and `add(int a, int b, int c)`
- By changing the data type of parameters: `add(int a, int b)` and `add(double a, double b)`
- By changing the order of parameters: `show(String name, int age)` and `show(int age, String name)`

Why Changing Only the Return Type Is Not Method Overloading

In Java, method calls are resolved by the compiler using the method's name and parameter list (its signature); the return type is not part of the method signature.

Difference between Method Overloading and Method Overriding

Basis	Method Overloading	Method Overriding
Definition	Same method name with different parameter lists in the same class	Subclass provides a specific implementation of a method already defined in its parent class
Polymorphism Type	Compile-time (static) polymorphism	Run-time (dynamic) polymorphism
Class Involved	Occurs within the same class	Occurs between parent and child classes (inheritance)
Parameters	Must be different (number/type/order)	Must be exactly the same
Return Type	Can be same or different	Must be same or a covariant type
Binding	Resolved at compile time (early binding)	Resolved at run time (late binding)
Purpose	Increases readability and code reusability	Achieves run-time behavior specific to subclass

Part B: Programming Questions

1. Calculator Addition

A class Calculator is created with the add() method overloaded three times: to add two integers, to add three integers, and to add two decimal numbers. An object of the class is created and all overloaded methods are called.

```
public class Calculator {  
  
    // Overloaded method to add two integers  
    int add(int a, int b) {  
        return a + b;  
    }  
  
    // Overloaded method to add three integers  
    int add(int a, int b, int c) {  
        return a + b + c;  
    }  
  
    // Overloaded method to add two decimal numbers  
    double add(double a, double b) {  
        return a + b;  
    }  
  
    public static void main(String[] args) {  
        Calculator calc = new Calculator();  
  
        System.out.println("Sum of two integers (5, 10): " + calc.add(5, 10));  
        System.out.println("Sum of three integers (5, 10, 15): " + calc.add(5, 10,  
15));  
        System.out.println("Sum of two decimals (5.5, 10.2): " + calc.add(5.5, 10.2));  
    }  
}
```

Output:

```
Sum of two integers (5, 10): 15  
Sum of three integers (5, 10, 15): 30  
Sum of two decimals (5.5, 10.2): 15.7
```

2. Display Method

A class Display is created with the show() method overloaded three times: to display only the student's name, to display the name and age, and to display the name, age, and marks.

```
public class Display {  
  
    // Overloaded method to display only student's name  
    void show(String name) {  
        System.out.println("Student Name: " + name);  
    }  
  
    // Overloaded method to display student's name and age
```

```

void show(String name, int age) {
    System.out.println("Student Name: " + name + ", Age: " + age);
}

// Overloaded method to display student's name, age, and marks
void show(String name, int age, double marks) {
    System.out.println("Student Name: " + name + ", Age: " + age + ", Marks: " +
marks);
}

public static void main(String[] args) {
    Display d = new Display();

    d.show("Akarsh Anand");
    d.show("Akarsh Anand", 20);
    d.show("Akarsh Anand", 20, 89.5);
}
}

```

Output:

```

Student Name: Akarsh Anand
Student Name: Akarsh Anand, Age: 20
Student Name: Akarsh Anand, Age: 20, Marks: 89.5

```

3. Find Maximum

A class FindMaximum is created with the max() method overloaded three times: to find the maximum between two integers, three integers, and two decimal numbers.

```

public class FindMaximum {

    // Overloaded method to find maximum of two integers
    int max(int a, int b) {
        return (a > b) ? a : b;
    }

    // Overloaded method to find maximum of three integers
    int max(int a, int b, int c) {
        int m = (a > b) ? a : b;
        return (m > c) ? m : c;
    }

    // Overloaded method to find maximum of two decimal numbers
    double max(double a, double b) {
        return (a > b) ? a : b;
    }

    public static void main(String[] args) {
        FindMaximum fm = new FindMaximum();

        System.out.println("Maximum of two integers (12, 25): " + fm.max(12, 25));
        System.out.println("Maximum of three integers (12, 25, 18): " + fm.max(12, 25,
18));
    }
}

```

```
        System.out.println("Maximum of two decimals (12.5, 9.8): " + fm.max(12.5,  
9.8));  
    }  
}
```

Output:

```
Maximum of two integers (12, 25): 25  
Maximum of three integers (12, 25, 18): 25  
Maximum of two decimals (12.5, 9.8): 12.5
```